

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación

Carrera de Ingeniería de Software

INSTRUMENTO DE EVALUACIÓN — INFORME TÉCNICO

GA-SUM-05 / PE-U4 — Práctica Experimental Unidad 4

Procesamiento Distribuido con Apache Spark: comprobación experimental de la Ley de Amdahl aplicada al Proyecto Fin de Curso

Asignatura: Aplicaciones Distribuidas (ISR-701)
Unidad: Unidad 4 — Cómputo Paralelo y Distribuido
Período académico: 2026–2027 PPA
Docente responsable: Gleiston C. Guerrero-Ulloa, M.Sc.

Equipo: D

PFC de referencia: ACC – Soporte Técnico ISP

Integrante			Rol	PFC de origen
Aucatoma	Celorio	Jhinson	Responsable de documentación	AGLS – TiendaTech
Stalyn	Alvarez	Parraga	Responsable de calidad	ACC – Soporte Técnico ISP
Jeremy	Alexis			
Carpio	Mendoza	Carlos José	Líder de desarrollo	ACC – Soporte Técnico ISP

Justificación técnica del PFC de referencia.

El equipo seleccionó el dominio **ACC – Soporte Técnico ISP**, orientado a la gestión de tickets de soporte técnico de un proveedor de servicios de Internet. Este tipo de sistema puede generar grandes volúmenes de información relacionados con incidencias de conectividad, velocidad, facturación, disponibilidad del servicio y ubicación geográfica. A medida que aumenta el número de clientes, el análisis histórico de estos tickets puede alcanzar cientos de miles o millones de registros, lo que permite evaluar el uso de herramientas de procesamiento distribuido para analizar la información de manera más eficiente.

La decisión de negocio que este trabajo busca apoyar es la **priorización automática de la cola de atención técnica**. A partir del tipo de incidencia, la ubicación y la prioridad asignada a cada ticket, un ISP podría identificar las zonas con mayor número de problemas, reforzar la asignación de personal técnico y decidir qué incidencias requieren una atención más rápida. Para representar este escenario se utilizó un conjunto de datos real y público de la Federal Communications Commission de Estados Unidos, cuya estructura presenta características comparables con los tickets que podría manejar un sistema de soporte técnico de un ISP.

Repositorio: <https://github.com/carlospatroner-boop/pe-u4-spark-Soporte-Tecnico-ISP>

Resumen

En esta práctica se evaluó el rendimiento de un proceso de análisis de datos implementado con pandas y Apache Spark, tomando como dominio de referencia el PFC ACC – Soporte Técnico ISP. Para el experimento se utilizó un conjunto de 600 000 registros reales de quejas de consumidores de servicios de telecomunicaciones, obtenido del portal público de la Federal Communications Commission (FCC). Sobre estos datos se implementaron cinco transformaciones equivalentes en pandas y PySpark: filtrado, agrupación, unión de datos, generación de una columna de prioridad y selección de los registros más recientes. Cada transformación fue ejecutada cinco veces y se utilizó la mediana de los tiempos obtenidos para realizar la comparación.

Los resultados mostraron que, para el volumen de datos y el entorno utilizados, pandas presentó menores tiempos de ejecución que PySpark en las cinco transformaciones. Sin embargo, al analizar el comportamiento de la transformación de unión con diferentes niveles de paralelismo, se comprobó que el aumento de recursos puede mejorar el rendimiento, aunque esta mejora se encuentra limitada por la parte del procesamiento que no escala de forma eficiente. El análisis mediante la Ley de Amdahl permitió identificar estas limitaciones y comprender que el uso de procesamiento distribuido no garantiza por sí mismo un mejor rendimiento. Para el contexto de Soporte Técnico ISP, los resultados indican que la adopción de Spark debe estar justificada por un volumen de información suficientemente grande o por necesidades de procesamiento continuo.

1. Introducción

El cómputo paralelo y distribuido dejó de ser una especialización de laboratorios de alto rendimiento para convertirse en infraestructura estándar de cualquier organización que procese datos a una escala en la que un único proceso secuencial ya no ofrece una latencia aceptable. Apache Spark es, en la actualidad, el motor de referencia para esta clase de procesamiento por su modelo de evaluación perezosa sobre RDDs y DataFrames, su tolerancia a fallos basada en *lineage* en vez de replicación, y su capacidad de unificar procesamiento por lotes, consultas SQL y aprendizaje automático sobre la misma abstracción [1]-[3]. Sin embargo, la ganancia de rendimiento que un motor distribuido aporta frente a un procesamiento secuencial no es automática ni universal: depende del volumen de datos, de la naturaleza de cada operación (si requiere o no *shuffle* entre particiones) y, de manera fundamental, de la fracción del trabajo total que efectivamente puede paralelizarse –el objeto de estudio de la Ley de Amdahl [4]–.

Esta práctica tiene como objetivo comprobar experimentalmente esa ley: implementar un *pipeline* idéntico en pandas (secuencial) y en PySpark (distribuido), medir sus tiempos de ejecución bajo un protocolo riguroso, y contrastar el *speedup* observado con el que predice el modelo teórico de Amdahl –y, complementariamente, con la reformulación de escalado de Gustafson-Barsis, más adecuada para interpretar el beneficio del paralelismo cuando el volumen de datos crece con el tiempo, como ocurre en la operación real de un ISP [5]–.

1.1. Objetivos

Objetivo general. Comprobar experimentalmente la Ley de Amdahl implementando y midiendo el rendimiento de un *pipeline* distribuido de datos con Apache Spark sobre un conjunto de datos real de al menos 500,000 registros, comparándolo con el procesamiento secuencial equivalente.

Objetivos específicos.

1. Implementar las mismas cinco transformaciones con pandas (secuencial) y con PySpark (distribuido).
2. Medir los tiempos de ejecución con precisión de microsegundos, con cinco repeticiones por transformación y reporte de la mediana.

3. Calcular el *speedup* experimental y compararlo con el *speedup* teórico de Amdahl.
4. Determinar la fracción serial de cada transformación y el límite práctico de paralelización.

El resto de este documento se organiza como sigue: la Section 2 desarrolla el marco teórico de paralelismo, MapReduce/Hadoop, arquitectura de Spark y computación en la nube; la Section 3 detalla el procedimiento aplicado; la Section 4 presenta los resultados cuantitativos con el trabajo matemático explícito del ajuste de Amdahl; la Section 5 discute críticamente estos resultados; el Anexo A (Section 6) responde las cuatro preguntas obligatorias de la guía; y la última sección recoge las conclusiones individuales de cada integrante del equipo.

2. Fundamento teórico

2.1. Fundamentos de paralelismo y Ley de Amdahl

2.1.1. Paralelismo de datos y paralelismo de tareas

Antes de introducir el modelo de Amdahl es necesario distinguir los dos paradigmas fundamentales bajo los cuales se descompone un problema computacional en unidades concurrentes. Grama et al. definen el *paralelismo de datos* como aquel que surge cuando operaciones idénticas se aplican de forma concurrente sobre elementos de datos distintos, mientras que el *paralelismo de tareas* es el que se expresa de forma natural mediante tareas independientes organizadas en un grafo de dependencias [6]. Un ejemplo canónico del primer caso es la multiplicación de matrices densas, donde cuatro tareas idénticas se aplican sobre submatrices distintas de los operandos; un ejemplo del segundo caso es el *quicksort* paralelo o la factorización de matrices dispersas, donde la estructura de dependencias entre subtareas —y no la uniformidad de la operación— determina el grado de concurrencia explotable [6].

Esta distinción no es meramente taxonómica: el propio grado de paralelismo de datos tiende a crecer con el tamaño del problema, lo que permite emplear más unidades de procesamiento a medida que el volumen de datos aumenta; sin embargo, ese crecimiento del *speedup* termina saturándose como consecuencia directa de la Ley de Amdahl [4], [6], que se desarrolla a continuación.

2.1.2. Ley de Amdahl: formulación y límite teórico

Amdahl argumentó en 1967 que todo programa contiene una fracción de trabajo inherentemente secuencial —en su caso, la sobrecarga de gestión de datos (*data management housekeeping*)— que no es susceptible de paralelización, y que dicha fracción, no el número de procesadores disponibles, es la que termina acotando el rendimiento máximo alcanzable [4]. El propio autor ilustra el argumento con una cifra empírica: si aproximadamente el 40 % de las instrucciones de una carga de producción corresponden a esa gestión secuencial, incluso reduciendo esa sobrecarga a la mitad mediante hardware dedicado, el límite superior de throughput queda fijado entre cinco y siete veces la tasa de procesamiento secuencial, sin importar cuántos procesadores adicionales se añadan [4].

Formalmente, sea p la fracción paralelizable de una tarea, $(1 - p)$ su fracción estrictamente serial y N el número de unidades de procesamiento. El *speedup* de Amdahl es

$$S(N) = \frac{1}{(1 - p) + \frac{p}{N}}, \quad (1)$$

cuyo límite superior, cuando $N \rightarrow \infty$, queda fijado exclusivamente por la fracción serial:

$$S_{\text{máx}} = \lim_{N \rightarrow \infty} S(N) = \frac{1}{1 - p}. \quad (2)$$

La ecuación (2) es la formalización directa del argumento de Amdahl: por más que N crezca sin límite, el segundo término del denominador de (1) tiende a cero y el speedup queda acotado por la parte no paralelizable del programa.

Finalmente, la eficiencia del paralelismo relaciona el speedup obtenido con los recursos invertidos para alcanzarlo:

$$E(N) = \frac{S(N)}{N}. \quad (3)$$

Grama et al. formalizan el speedup, en términos generales, como el cociente entre el tiempo de ejecución del mejor algoritmo secuencial conocido y el tiempo del algoritmo paralelo equivalente sobre N elementos de procesamiento, y muestran experimentalmente que dicho speedup satura y la eficiencia decae precisamente como consecuencia de la Ley de Amdahl [4], [6].

2.1.3. Ley de Gustafson-Barsis: escalado en lugar de tamaño fijo

Gustafson señaló en 1988 que la formulación de Amdahl asume implícitamente que la fracción paralelizable p es independiente de N , una condición que rara vez se cumple en la práctica: en los entornos de producción reales el tamaño del problema tiende a escalar junto con la capacidad de cómputo disponible, de modo que resulta más realista mantener constante el tiempo de ejecución que el tamaño del problema [5]. A partir de esta observación propone el *speedup escalado*, que en su forma simplificada se expresa como

$$S_G(N) = N - (1 - p)(N - 1). \quad (4)$$

A diferencia de la curva pesimista de (1), la ecuación (4) crece casi linealmente con N , porque supone que la porción de trabajo paralelizable aumenta junto con los recursos disponibles en lugar de permanecer fija. Gustafson respalda el argumento con tres aplicaciones reales ejecutadas en un hipercubo de 1024 procesadores en Sandia National Laboratories —análisis de esfuerzo en vigas, simulación de ondas superficiales y flujo de fluido inestable—, en las que se obtuvieron factores de speedup superiores a 1000, muy por encima de lo que la Ley de Amdahl predeciría para fracciones seriales de apenas 0.4–0.8 % [5].

2.1.4. Forma inversa de la fracción serial: la métrica de Karp-Flatt

Las ecuaciones (1) y (4) permiten predecir el speedup a partir de una fracción serial conocida, pero no resuelven el problema inverso: estimar esa fracción a partir de mediciones experimentales de speedup. Karp y Flatt abordan esta limitación proponiendo la *fracción serial experimentalmente determinada*, derivada algebraicamente de la Ley de Amdahl [7]. Despejando p de (1) para una medición con N unidades y speedup observado S se obtiene

$$p = \frac{\frac{1}{S} - \frac{1}{N}}{1 - \frac{1}{N}}, \quad N > 1. \quad (5)$$

Karp y Flatt argumentan que esta métrica es más informativa que el speedup o la eficiencia por separado porque, en un sistema ideal, debería permanecer aproximadamente constante al variar N ; cualquier desviación sistemática de esa constancia delata sobrecostos que ni el speedup ni la eficiencia revelan por sí solos, como el desbalance de carga entre procesadores, la sobrecarga de sincronización, la contención de memoria o la reducción del tamaño efectivo de los vectores al fragmentar el trabajo [7]. Los propios autores ilustran el poder diagnóstico de la métrica con datos de un Cray Y-MP/8, donde la fracción serial se mantiene prácticamente constante (en torno a 0.021–0.024) para distintos números de procesadores, lo que permite concluir que la pérdida de eficiencia observada se debe al paralelismo intrínsecamente limitado del software y no a sobrecostos adicionales del sistema [7].

Advertencia metodológica. La magnitud que se despeja mediante (5) no es la fracción serial algorítmica —es decir, la porción de código que un análisis estático del algoritmo revelaría como no paralelizable—, sino la *fracción no escalable observada*, que agrega además la sobrecarga real de planificación, serialización y *shuffle* introducida por el motor de ejecución [7]. Confundir ambas magnitudes es un error conceptual: una fracción serial observada alta puede deberse tanto a limitaciones intrínsecas del algoritmo como a overhead de infraestructura, y solo un análisis diagnóstico como el propuesto por Karp y Flatt permite distinguir entre ambas causas.

2.1.5. Representación gráfica del speedup

La Figura 1 contrasta la curva de speedup (1) frente al número de procesadores N para fracciones paralelizables $p = 0,5, 0,75, 0,9$ y $0,95$, generada con `matplotlib` a partir del script `src/graficas.py`. Se observa que, incluso con una fracción paralelizable tan alta como $p = 0,95$, el speedup se satura alrededor de $S_{\text{máx}} = 1/(1 - 0,95) = 20$, lo que ilustra de forma directa el límite impuesto por la ecuación (2).

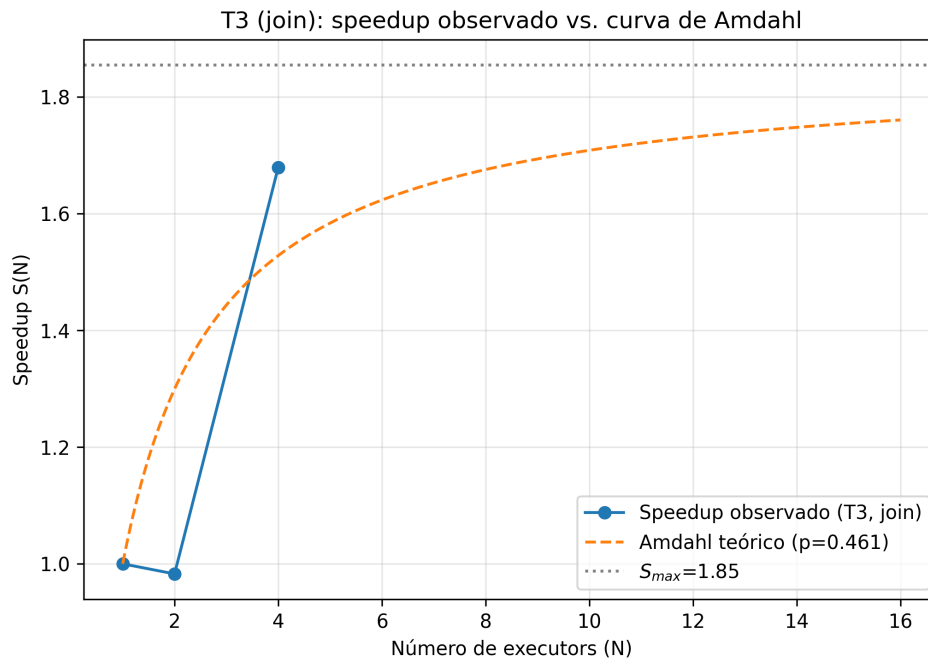


Figura 1: Speedup teórico según la Ley de Amdahl para $p = 0,5, 0,75, 0,9, 0,95$ frente al número de procesadores N .

2.1.6. Limitaciones para el procesamiento iterativo

La misma restricción que simplifica el modelo de programación impone, sin embargo, una limitación estructural: MapReduce materializa los resultados intermedios en disco al final de cada job y debe volver a leerlos desde disco en el job siguiente, lo cual penaliza fuertemente a los algoritmos iterativos —comunes en aprendizaje automático y en análisis de grafos— que necesitan reutilizar repetidamente el mismo conjunto de datos en memoria entre iteraciones [8]. Esta limitación fue, de hecho, uno de los motivadores explícitos del rediseño de Hadoop hacia YARN, cuyos autores señalan que, bajo la arquitectura original, los usuarios recurrían a patrones forzados —como enviar trabajos de *solo map* para simular cómputo iterativo con planificación en grupo— precisamente porque el modelo MapReduce clásico no ofrecía un mecanismo nativo para este tipo de cargas [8], [9].

2.1.7. Evidencia empírica: Hadoop frente a Spark

Cualquier afirmación comparativa de rendimiento entre Hadoop y Spark debe apoyarse en evidencia empírica publicada y no en material promocional de proveedores. Ahmed et al. realizan una comparación sistemática de ambos motores mediante la suite de *benchmarking* HiBench, ejecutando cargas de trabajo de tipo **WordCount** (agregación) y **TeraSort** (*shuffle* intensivo) sobre volúmenes de hasta 600 GB en un clúster de 80 núcleos y 60 TB de almacenamiento [10]. Sus resultados muestran que Spark supera a Hadoop en ambas cargas cuando los parámetros se sintonizan correctamente, con mejoras de hasta 2× en **WordCount** y de hasta 14× en **TeraSort** para los volúmenes de datos evaluados, aunque el rendimiento de Spark se degrada cuando el tamaño de los datos excede la memoria disponible [10]. Esta ventaja de Spark se explica principalmente porque procesa los conjuntos de trabajo en memoria en lugar de materializarlos en disco entre etapas, evitando el ciclo de lectura/escritura que penaliza a MapReduce, una explicación consistente con la limitación estructural descrita en la subsección anterior [8], [10].

2.2. Apache Spark: arquitectura y modelo de programación

2.2.1. Arquitectura: driver, cluster manager y executors

La arquitectura de Spark se organiza en torno a tres componentes. El **driver** ejecuta el programa principal de la aplicación, construye el plan lógico de ejecución y solicita los recursos necesarios al **cluster manager** —que puede operar en modo autónomo (*standalone*), sobre YARN (véase la Sección 2.1.6) o sobre Kubernetes—, el cual reparte esos recursos entre las aplicaciones activas del clúster [11], [12]. Los **executors** son los procesos que efectivamente resuelven las tareas asignadas en los nodos trabajadores: almacenan datos en memoria o disco y reportan periódicamente su estado al **driver**. Cada aplicación recibe su propio conjunto de executors, que persisten durante toda su ejecución; esto permite reutilizar datos cacheados entre etapas de una misma aplicación, pero impide compartirlos directamente con otras aplicaciones sin recurrir a un almacenamiento externo [11], [12].

2.2.2. El modelo RDD y la tolerancia a fallos mediante lineage

El modelo de *Resilient Distributed Dataset* (RDD) resuelve el problema de la tolerancia a fallos sin recurrir a la replicación física de los datos entre nodos, a diferencia de la durabilidad que HDFS provee a MapReduce entre etapas [1]. En su lugar, cada RDD mantiene un *lineage*: un registro de las transformaciones que lo originaron a partir de datos fuente u otros RDD. Ante la pérdida de una partición por fallo de un nodo, Spark recorre ese lineage y recalcula únicamente la partición afectada, en lugar de tener que recuperar una réplica completa desde disco [1], [11]. Esta estrategia reduce el costo de tolerancia a fallos en cargas iterativas, aunque tiene una contrapartida: cuando las cadenas de lineage se alargan demasiado —por ejemplo, tras muchas iteraciones sucesivas—, el costo de recomputación ante un fallo también crece, lo que hace conveniente introducir puntos de control (*checkpoints*) que trunquen el lineage de forma periódica [1].

2.2.3. DataFrames, Datasets y evaluación perezosa

Sobre los RDD se construyen las APIs de DataFrames y Datasets, que añaden un esquema de columnas nombradas y permiten a Spark aprovechar el optimizador Catalyst para planificar la ejecución; el Dataset suma, además, verificación de tipos en tiempo de compilación [2], [3]. Todas las transformaciones sobre estas estructuras son perezosas (*lazy*): no se ejecutan en el momento en que se escriben, sino que se acumulan en un plan lógico hasta que una acción exige devolver un resultado concreto al **driver**. Ese diferimiento es precisamente lo que permite a Catalyst reordenar operaciones, descender predicados de filtrado y podar columnas antes de

ejecutar nada, y explica por qué el plan real de ejecución no sigue necesariamente el orden en que aparece escrito el código fuente [2], [3].

2.2.4. DAG, stages, tasks y comparación de rendimiento

Cuando una acción dispara la ejecución, Spark materializa el plan lógico como un grafo acíclico dirigido (DAG) que el planificador divide en *stages* según el tipo de dependencia entre transformaciones: las dependencias estrechas (*narrow*) se encadenan dentro de un mismo stage sin mover datos entre nodos, mientras que las dependencias amplias (*wide*) exigen un *shuffle* y abren un stage nuevo [3], [11]. Cada stage se descompone a su vez en *tasks* —una por partición—, que los executors resuelven en paralelo. La evidencia empírica sitúa a Spark, en promedio, alrededor de cinco veces más rápido que MapReduce gracias al procesamiento en memoria, aunque esa ventaja se reduce cuando el volumen de datos supera la memoria disponible del clúster, escenario en el que MapReduce conserva una eficiencia comparativamente mayor en tareas de menor tamaño [13], [14].

2.3. Computación en la nube: modelos y proveedores

2.3.1. Definición del NIST y características esenciales

El NIST define la computación en la nube como un modelo que habilita el acceso ubicuo, conveniente y bajo demanda a un conjunto compartido de recursos de cómputo configurables —redes, servidores, almacenamiento, aplicaciones y servicios— que pueden aprovisionarse y liberarse con un esfuerzo mínimo de gestión o interacción con el proveedor [15]. Ese modelo se compone de cinco características esenciales: autoservicio bajo demanda, acceso amplio a la red, agrupación de recursos (*resource pooling*), elasticidad rápida y servicio medido; además de tres modelos de servicio y cuatro modelos de despliegue que la propia especificación detalla y que sirven de vocabulario estándar para describir cualquier oferta comercial de nube [15], [16].

2.3.2. Modelos de servicio: IaaS, PaaS, SaaS y FaaS

Los tres modelos de servicio definidos por el NIST se distinguen por el grado de responsabilidad de gestión que asume el consumidor frente al proveedor. Infraestructura como Servicio (IaaS) entrega capacidad de cómputo, almacenamiento y red en bruto, sobre la cual el usuario despliega su propio sistema operativo y pila de software; Plataforma como Servicio (PaaS) ofrece un entorno de ejecución ya gestionado en el que el consumidor solo aporta la aplicación; y Software como Servicio (SaaS) entrega software ya operativo, sin que el consumidor gestione infraestructura ni plataforma alguna [15]. *Function as a Service* (FaaS) no forma parte de la definición original del NIST de 2011, sino que constituye una evolución posterior del modelo PaaS en la que el proveedor ejecuta fragmentos de código por invocación individual y factura únicamente el tiempo de cómputo efectivamente consumido, sin que el consumidor gestione servidores ni instancias de forma explícita [15], [17].

2.3.3. Proveedores para procesamiento distribuido

Los tres grandes proveedores de nube pública ofrecen servicios gestionados equivalentes en propósito para ejecutar motores como Hadoop y Spark. Amazon EMR es una plataforma de clústeres gestionados que simplifica el aprovisionamiento y la operación de estos motores, con variantes de despliegue sobre EC2, sobre Kubernetes (EMR on EKS) y sin servidor (EMR Serverless); Azure HDInsight cumple una función equivalente dentro del ecosistema Microsoft, aunque conviene verificar el estado de soporte de cada versión antes de dimensionar un despliegue, dado que Microsoft ha retirado versiones y variantes del servicio en el pasado [16]. En Google Cloud, el

servicio conocido históricamente como Dataproc pasó a denominarse *Managed Service for Apache Spark*, marca que agrupa tanto el despliegue en clúster tradicional como el despliegue sin servidor, en el que la infraestructura escala automáticamente y solo se factura el tiempo en que la carga se ejecuta [16], [17]. Cualquier afirmación sobre costo o dimensionamiento de recursos en el dominio del PFC debe contrastarse con la documentación oficial vigente del proveedor elegido, consultada en una fecha específica, dado que las tarifas y nomenclaturas de estos servicios cambian con frecuencia.

2.3.4. Arquitecturas Lambda y Kappa

Sobre esta infraestructura de proveedores se apoyan dos patrones arquitectónicos para organizar el procesamiento de datos a gran escala. La arquitectura Lambda separa el flujo de datos en tres capas: una capa *batch* que procesa el histórico completo con máxima exactitud pero alta latencia, una capa de velocidad (*speed layer*) que procesa los datos más recientes para ofrecer resultados de baja latencia, y una capa de servicio que unifica ambas vistas para responder a las consultas del usuario [18], [19]. La arquitectura Kappa elimina la capa *batch* y trata todo dato como un flujo continuo sobre una única canalización de *streaming*, lo que reduce la duplicación de lógica entre las capas batch y de velocidad a costa de depender por completo de la robustez y capacidad de reprocesamiento del motor de *streaming* elegido [18], [19].

3. Procedimiento aplicado

3.1. Preparación del entorno y del conjunto de datos

Dado que PySpark en Windows requiere `winutils.exe` y librerías nativas de Hadoop frágiles de instalar, el equipo ejecutó todo el *pipeline* dentro de un contenedor Docker Linux (`eclipse-temurin:21-jdk-jammy` con Java 21, Python 3.10, PySpark 4.1.2, pandas, numpy, matplotlib, scipy y Jupyter), garantizando que el entorno sea idéntico en cualquier máquina del equipo.

El conjunto de datos se extrajo de *CGB — Consumer Complaints Data* de la FCC (<https://opendata.fcc.gov/d/3xyp-aqkj>), un portal de datos abiertos gubernamental con licencia **Public Domain (U.S. Government Work)**. El portal expone 3,599,354 registros totales (verificado el día de la descarga con `$select=count(*)` sobre el mismo *endpoint*); de ellos se extrajeron los **600,000 tickets más recientes** restringidos a las tres categorías directamente relacionadas con servicios de un ISP (*Phone, Internet, TV*), vía la API pública SODA (Socrata Open Data API), paginada en bloques de 50,000 filas:

```
1 GET https://opendata.fcc.gov/resource/3xyp-aqkj.csv
2 ?$select=id,ticket_created,issue_type,method,issue,city,state,zip
3 &$where=issue_type in('Phone','Internet','TV')
4 &$order=ticket_created DESC
5 &$limit=50000&$offset={0,50000,...,550000}
```

Listing 1: Extracción del dataset vía la API SODA de la FCC (paginada en bloques de 50,000 filas).

El extracto resultante tiene 600,000 filas, 8 columnas, 69 MB en disco, y abarca el rango de fechas del 2024-09-06 al 2026-08-03. Contiene valores nulos reales (no inducidos) en `method` (15,466), `issue` (2,456), `city` (694), `state` (181) y `zip` (192), que se aprovechan como condición de filtrado en T1. El detalle completo —incluyendo la distribución por categoría y por estado— se documenta en `data/README_dataset.md` del repositorio. El conjunto se cargó y verificó tanto en pandas (`pd.read_csv`) como en un DataFrame de PySpark (`spark.read.csv` con `inferSchema`), confirmando en ambos motores 600,000 filas y el mismo esquema inferido.

3.2. Las cinco transformaciones

Las cinco transformaciones exigidas por la guía se implementaron de forma idéntica –misma lógica de negocio, mismas condiciones– en pandas y en PySpark, reutilizando en ambos motores el mismo módulo de referencia (`src/referencia.py`) para la tabla de regiones del *join* (T3) y la función de clasificación de prioridad (T4), de modo que la verificación de equivalencia entre motores (Section 4) sea válida y no dependa de una coincidencia accidental.

```

1 def t1_filtrado(df):
2     return df.filter(
3         F.col("issue_type").isin("Internet", "Phone")
4         & F.col("method").isNotNull()
5         & F.col("city").isNotNull()
6         & F.col("state").isNotNull()
7     ).select(*COLUMNAS_T1)
8
9 def t2_agrupacion(df):
10    return df.groupBy("state").agg(
11        F.count("id").alias("total_tickets"),
12        F.countDistinct("issue").alias("tipos_issue_distintos"),
13        F.min("ticket_created").alias("primer_ticket"),
14        F.max("ticket_created").alias("ultimo_ticket"),
15    )
16
17 def t3_join(df, regiones):
18    return df.join(regiones, on="state", how="left")
19
20 def t4_columna_derivada(df):
21    return df.withColumn("prioridad", _prioridad_udf(F.col("issue_type"), F.col(
22        "issue")))
23
24 def t5_top_n(df, n=1000):
25    return df.orderBy(F.col("ticket_created").desc()).limit(n)

```

Listing 2: Las cinco transformaciones en PySpark (`src/transformaciones_spark.py`).

T1 (filtrado): tickets de las categorías *Internet* o *Phone* con `method`, `city` y `state` no nulos –condición compuesta– con selección de siete columnas relevantes. **T2 (agrupación):** agrupación por estado con cuatro funciones de agregación (`count`, `countDistinct`, `min`, `max`). **T3 (*join*):** unión por `state` entre el DataFrame de tickets y una segunda tabla de dimensión con la región censal de EE. UU. de cada estado (56 filas), construida en código a partir de la clasificación oficial de la Oficina del Censo de EE. UU. **T4 (columna derivada):** una función definida por el usuario (`clasificar_prioridad`, en `referencia.py`) que combina `issue_type` e `issue` para asignar una prioridad de atención técnica (ALTA para incidencias de disponibilidad/interferencia/velocidad, MEDIA para facturación/cuenta, BAJA para quejas de contenido), registrada como UDF de Spark y aplicada con `.apply()` en pandas. **T5 (*top-N*):** ordenamiento por fecha de apertura descendente y selección de los 1,000 tickets más recientes.

3.3. Protocolo de medición

Cada transformación se midió con `time.perf_counter()` bajo un protocolo común (`src/medicion.py`): una ejecución de calentamiento descartada explícitamente, seguida de cinco repeticiones cronometradas, reportando la mediana.

```

1 def medir(transformacion, func, reps=5, warmup=1, materialize=None):
2     for _ in range(warmup):
3         r = func()
4         if materialize is not None:
5             materialize(r)
6     tiempos = []

```

```

7   for _ in range(reps):
8       t0 = time.perf_counter()
9       resultado = func()
10      if materialize is not None:
11          materialize(resultado)
12      t1 = time.perf_counter()
13      tiempos.append(t1 - t0)
14  return tiempos, statistics.median(tiempos), resultado

```

Listing 3: Protocolo de medición común a pandas y PySpark (`src/medicion.py`).

Para PySpark, dado que las transformaciones son perezosas, cada medición fuerza la materialización del resultado con la acción `count()` antes de detener el cronómetro, evitando medir únicamente la construcción del DAG. La sesión de Spark se inicializó con `SparkSession.builder.config("spark.executor.cores", "4").getOrCreate()` y la configuración efectiva se documentó con `spark.sparkContext.getConf().getAll()` para cada una de las sesiones usadas (`resultados/spark_config_efectiva_*.json`). Es importante notar, con honestidad metodológica, que en modo `local[N]` —el único disponible dado que el equipo no contó con un clúster físico multinodo— el parámetro `spark.executor.instances` no tiene efecto real sobre el paralelismo; el grado de paralelismo se controla mediante el número de hilos de la URL maestra (`local[N]`), que sí se documenta y varía explícitamente en el escalado de T3.

Para la transformación T3 (*join*), además de la medición en la configuración por defecto, se repitió el protocolo completo con `local[1]`, `local[2]` y `local[4]` —reiniciando la sesión de Spark en cada caso, dado que el número de hilos del maestro no puede cambiarse sin reiniciar el `SparkContext`— para habilitar el análisis de escalabilidad y el ajuste de la Ley de Amdahl (Section 4). La equivalencia de resultados entre pandas y PySpark se verificó para las cinco transformaciones comparando cardinalidad y agregados de control (distribución de categorías, sumas de conteos, primer resultado tras ordenar), documentado en `resultados/equivalencia.json`.

4. Resultados

4.1. Tiempos de ejecución y *speedup* por transformación

La Table 1 resume, para cada transformación, la mediana de cinco repeticiones (descartada la ejecución de calentamiento) en pandas y en PySpark (sesión `local[4]`), y el *speedup* $S = T_{\text{pandas}}/T_{\text{PySpark}}$.

Tabla 1: Tiempos de ejecución (mediana de 5 repeticiones) y *speedup* por transformación. Los tiempos crudos completos están en `resultados/tiempos_crudos.csv`.

Transformación	T_{pandas} (s)	T_{PySpark} (s)	Speedup S
T1 — Filtrado	0,1846	1,0789	0,1711
T2 — Agrupación	0,1575	1,8854	0,0835
T3 — <i>Join</i>	0,1363	1,8290	0,0745
T4 — Columna derivada	0,3559	0,8483	0,4195
T5 — <i>Top-N</i>	0,1169	1,4286	0,0818

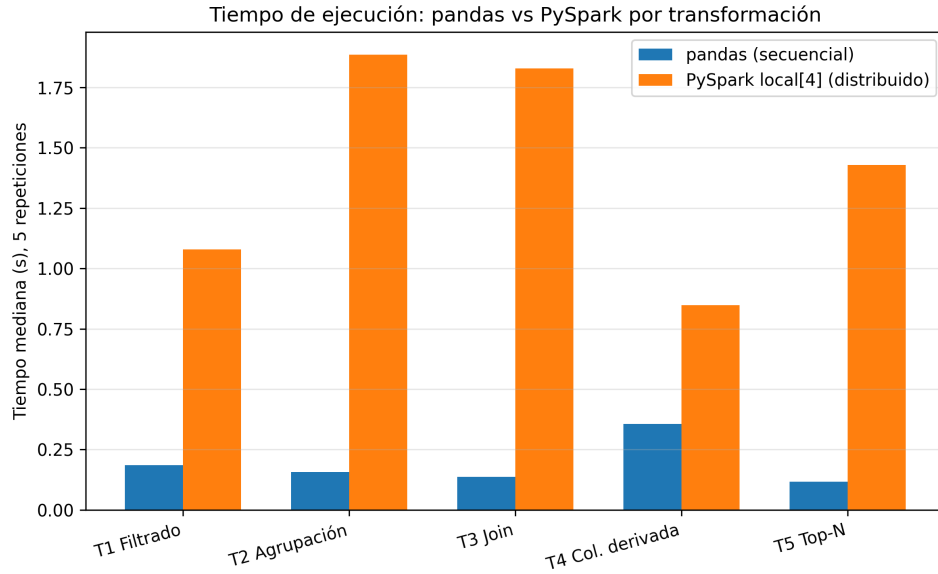


Figura 2: Tiempo de ejecución (mediana de 5 repeticiones) de pandas frente a PySpark (`local[4]`) para cada una de las cinco transformaciones.

El hallazgo central de esta tabla –y de la Figure 2– es que, a esta escala de datos (600,000 filas), **PySpark resulta más lento que pandas en las cinco transformaciones**, con *speedups* entre 0.075 y 0.42. Este resultado no es un error de medición: se discute en profundidad, con su causa y su umbral de rentabilidad, en la Pregunta 2 del Anexo A (Section 6) y en la Section 5. En resumen adelantado, la sobrecarga fija de un motor distribuido –arranque de la JVM, planificación de *tasks* por el driver, serialización Python $\leftrightarrow$ JVM vía Py4J, y para T3 específicamente la construcción de un plan de *Adaptive Query Execution*– domina sobre el tiempo de cómputo real cuando el volumen de datos es de cientos de miles de filas en vez de decenas o cientos de millones. La transformación T4 (columna derivada) es la que menos penaliza a Spark proporcionalmente ($S = 0.42$) porque es la única que ejecuta, por fila, más trabajo de cómputo efectivo (la evaluación de la UDF de prioridad) en relación con su overhead fijo, mientras que T2 y T3 –predominantemente coordinación y *shuffle*– son las más penalizadas.

4.2. Escalado de *executors* y ajuste de la Ley de Amdahl (T3)

La Table 2 presenta la mediana de tiempos de la transformación T3 (*join*) con 1, 2 y 4 *executors* (`local[N]`), y el *speedup* observado respecto de $N = 1$.

Tabla 2: Escalado de T3 (*join*) con 1, 2 y 4 *executors*.

N	$T_{T3}(N)$ (s)	Speedup $S(N)$
1	3,0712	1,0000
2	3,1252	0,9827
4	1,8290	1,6792

4.2.1. Cálculo de la fracción serial observada

Sustituyendo $N = 4$ y $S(4) = 1,6792$ en la forma inversa de la Ley de Amdahl (Equation (5)):

$$\begin{aligned}
p &= \frac{\frac{1}{S(4)} - \frac{1}{4}}{1 - \frac{1}{4}} = \frac{\frac{1}{1,6792} - 0,25}{0,75} \\
&= \frac{0,5956 - 0,25}{0,75} = \frac{0,3456}{0,75} = 0,4607.
\end{aligned} \tag{6}$$

Como verificación de robustez, se ajustó además p por mínimos cuadrados no lineales sobre los tres puntos $(N, S(N))$ simultáneamente (función `scipy.optimize.curve_fit` sobre la Equation (1)), obteniendo $p_{\text{ajustado}} = 0,4600$ –a menos de 0.2 % del valor algebraico puntual de la Equation (6), lo que confirma que el punto $N = 4$ por sí solo ya es representativo del ajuste global.

Cabe señalar, con la misma sustitución pero usando $N = 2$ y $S(2) = 0,9827$:

$$p_{N=2} = \frac{\frac{1}{0,9827} - \frac{1}{2}}{1 - \frac{1}{2}} = \frac{1,0176 - 0,5}{0,5} = 1,0353, \tag{7}$$

un valor matemáticamente inválido (p no puede superar 1). Esto no es un error de cálculo sino la manifestación directa de la advertencia metodológica de la Section 2.1: la p despejada de la forma inversa no es la fracción serial algorítmica, sino la fracción no escalable *observada*, y a $N = 2$ el *speedup* medido (0,9827) es en realidad *menor* que a $N = 1$ –el paso de 1 a 2 hilos en `local[N]` añade coordinación (planificación de dos *tasks*, sincronización) sin beneficio neto medible en una máquina de pocos núcleos físicos, un efecto de sobrecarga que domina completamente sobre cualquier ganancia real de paralelismo en ese tramo–. Por esta razón se reporta $p = 0,4607$ (obtenido con $N = 4$, el punto de mayor paralelismo real medido) como el valor principal, y se documenta $p_{N=2}$ únicamente como evidencia de la variabilidad de la medición a bajo N , consistente con lo señalado por Jain sobre la necesidad de tratar mediciones de sistemas como variables aleatorias y no como valores deterministas.

4.2.2. Speedup máximo teórico y N para el 90 % de $S_{\text{máx}}$

Con $p = 0,4607$, el speedup máximo teórico (Equation (2)) es

$$S_{\text{máx}} = \frac{1}{1-p} = \frac{1}{1-0,4607} = \frac{1}{0,5393} = 1,8543. \tag{8}$$

Es decir, sin importar cuántos *executors* adicionales se agreguen, la transformación T3 no podría acelerarse más de $1,85\times$ respecto de su ejecución en un único hilo, dada la fracción no escalable observada. Para determinar cuántos *executors* se necesitan para alcanzar el 90 % de ese máximo, se despeja N de $S(N) = 0,9 \cdot S_{\text{máx}}$ a partir de la Equation (1):

$$\begin{aligned}
0,9 S_{\text{máx}} &= \frac{1}{(1-p) + \frac{p}{N}} \implies (1-p) + \frac{p}{N} = \frac{1}{0,9 S_{\text{máx}}} \\
\frac{p}{N} &= \frac{1}{0,9 S_{\text{máx}}} - (1-p) \implies N = \frac{p}{\frac{1}{0,9 S_{\text{máx}}} - (1-p)}.
\end{aligned} \tag{9}$$

Sustituyendo $p = 0,4607$ y $S_{\text{máx}} = 1,8543$:

$$N = \frac{0,4607}{\frac{1}{0,9 \times 1,8543} - 0,5393} = \frac{0,4607}{0,5990 - 0,5393} = \frac{0,4607}{0,0599} \approx 7,69. \tag{10}$$

Se necesitarían, por tanto, **aproximadamente 8 *executors*** para que la transformación T3 alcance el 90 % de su *speedup* máximo teórico –un resultado que se retoma textualmente en la Pregunta 1 del Anexo A–.

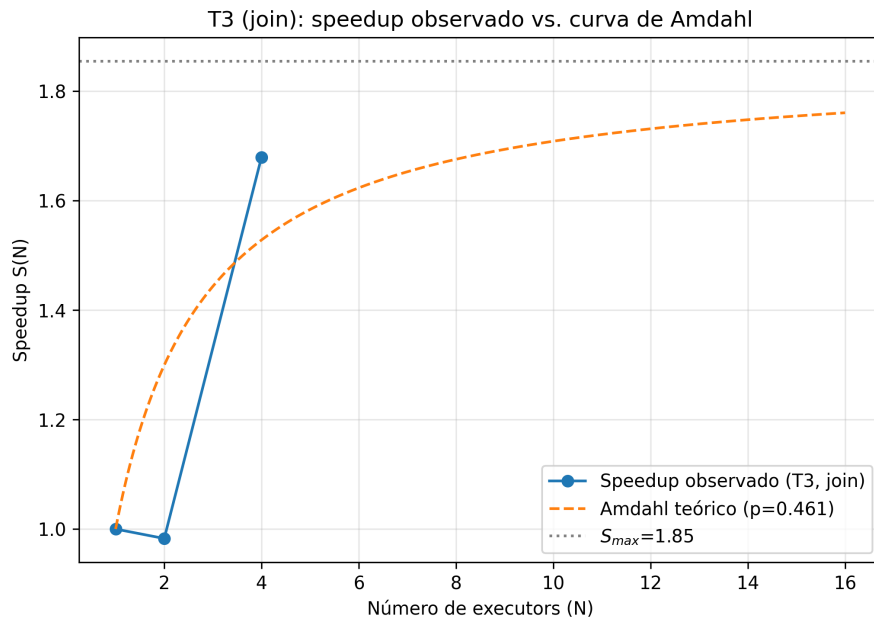


Figura 3: Speedup observado de T3 (*join*) frente al número de *executors*, superpuesto con la curva teórica de Amdahl ajustada ($p = 0,4607$) y su asíntota $S_{\text{máx}} = 1,8543$.

La Figure 3 hace visible el comportamiento no monótono entre $N = 1$ y $N = 2$ discutido arriba, y cómo el punto $N = 4$ –el único con paralelismo real efectivo en esta máquina de 4 núcleos físicos– se ajusta razonablemente a la curva teórica extrapolada desde ese único punto informativo.

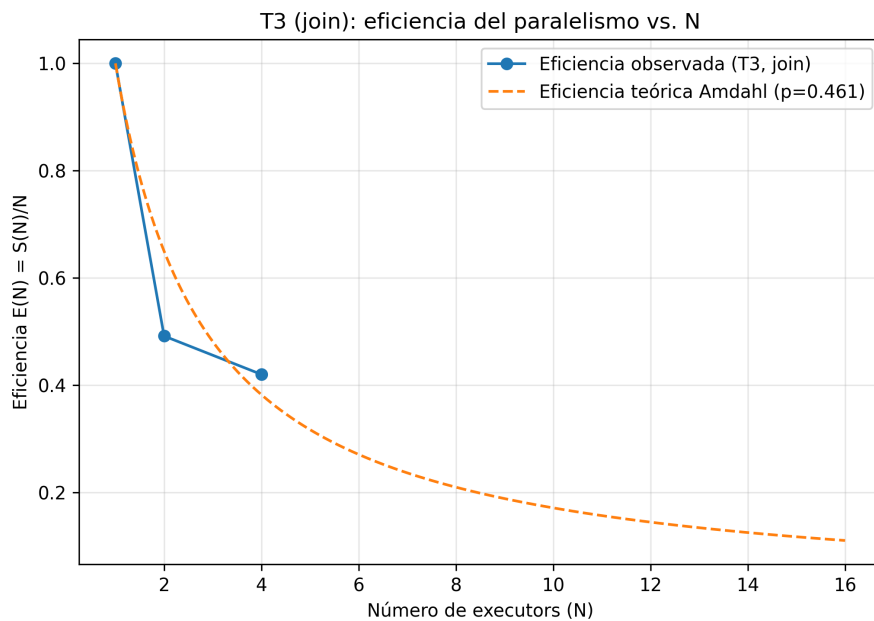


Figura 4: Eficiencia del paralelismo $E(N) = S(N)/N$ observada y teórica para T3 (*join*).

La Figure 4 muestra que la eficiencia cae por debajo de $E(N) = 0,5$ ya en $N = 2$ (reflejando

el *speedup* sub-unitario en ese punto) y se recupera parcialmente en $N = 4$ ($E(4) = S(4)/4 = 1,6792/4 = 0,42$), pero se mantiene lejos del ideal $E = 1$: agregar el cuarto *executor* rinde menos de la mitad del beneficio proporcional que rendiría en un escenario sin sobrecarga, coherente con una fracción no escalable observada superior al 46 %.

4.3. Verificación de equivalencia entre motores

Se verificó que pandas y PySpark produjeran resultados equivalentes para las cinco transformaciones, comparando cardinalidad y agregados de control (no fila por fila, dado que el orden de salida de un motor distribuido no está garantizado salvo ordenamiento explícito). Las cinco transformaciones coincidieron **exactamente**: T1 (558,289 filas en ambos motores, misma distribución por *issue_type*), T2 (57 grupos, suma de *total_tickets* igual a 600,000 en ambos), T3 (600,000 filas, misma distribución por región censal), T4 (600,000 filas, misma distribución de prioridad: 276,702 BAJA / 180,288 MEDIA / 143,010 ALTA) y T5 (1,000 filas, mismo primer *id* tras ordenar). El detalle completo está en `resultados/equivalencia.json`.

4.4. Evidencia de ejecución: Spark UI de la transformación T3

Details for Job 2

Status: SUCCEEDED

Submitted: 2026/08/04 02:42:09

Duration: 2 s

Associated SQL Query: 2

Completed Stages: 1

► Event Timeline

▼ DAG Visualization

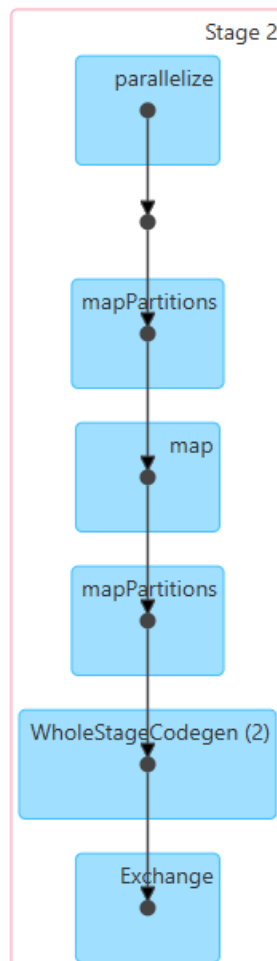


Figura 5: Spark UI — DAG Visualization del *job* correspondiente a la transformación T3 (*join*), mostrando el *stage* 2 con las etapas `parallelize` → `mapPartitions` → `map` → `mapPartitions` → `WholeStageCodegen` → `Exchange`.

La Figure 5 confirma empíricamente un punto discutido en la Section 2.2: el plan físico real de T3 no es un simple *join* de dos lecturas, sino un DAG con múltiples etapas de `WholeStageCodegen` (compilación de código Java en tiempo de ejecución, una optimización de Spark SQL que fusiona varias operaciones en un solo bloque compilado) y un nodo `Exchange` correspondiente al *shuffle* de la tabla de regiones antes del *join*. La inspección del plan completo vía la pestaña *SQL / Data-Frame* de la Spark UI (documentada en `evidencia/spark_ui_t3_stages.png`) confirma además que el optimizador Catalyst incorpora nodos de *Adaptive Query Execution* (`AQEShuffleRead`) que ajustan el plan de *shuffle* en tiempo de ejecución según el tamaño real observado de cada

partición –el mismo tipo de decisión adaptativa mencionado en la Section 2.2–, y que el nodo final `BroadcastHashJoin` confirma que Spark decidió difundir la tabla de regiones (56 filas) en lugar de aplicar un *shuffle* simétrico sobre ambos lados del *join*.

5. Análisis crítico

Los resultados de la Section 4 obligan a matizar la premisa con la que muchos equipos abordan un ejercicio de «comprobar la Ley de Amdahl con Spark»: la ley predice un límite superior al *speedup* alcanzable, no garantiza que ese *speedup* sea positivo en primer lugar. A 600,000 filas, la fracción no escalable observada ($p \approx 0,46$) no proviene principalmente de un cuello de botella algorítmico dentro de las transformaciones –T1 es un filtro trivial, T5 es un ordenamiento sobre una columna indexable– sino de la sobrecarga fija de coordinar un motor distribuido: arranque de la JVM y de la sesión de Spark (medido, pero fuera del cronómetro de cada transformación individual), planificación de *tasks* por el driver, serialización de datos entre el proceso Python y la JVM a través de Py4J, y para T2/T3 específicamente, el costo de *shuffle* incluso cuando –como reveló la evidencia de Spark UI– Spark opta por difundir la tabla pequeña en vez de mezclar ambos lados. Esta sobrecarga es esencialmente *constante* por transformación, independiente del volumen de datos dentro de un rango razonable, mientras que el tiempo de pandas crece linealmente con las filas; la consecuencia directa es que existe un volumen de datos de cruce por debajo del cual pandas gana y por encima del cual Spark gana, discutido cuantitativamente en la Pregunta 2 del Anexo A.

Un segundo punto crítico es la limitación del entorno de medición. El equipo no dispuso de un clúster Spark multinodo real, sino de `local[N]` sobre una única máquina con recursos limitados (documentado en la configuración efectiva de cada sesión). Esto tiene dos consecuencias metodológicas que deben declararse con honestidad: primero, el parámetro `spark.executor.instances` exigido por la guía se configuró pero *no tiene efecto real* en modo local, donde el paralelismo depende exclusivamente del número de hilos de `local[N]`; segundo, con solo tres puntos de medición ($N = 1, 2, 4$) y una anomalía de sobrecarga entre $N = 1$ y $N = 2$, el ajuste de p se apoya en la práctica en un único punto informativo ($N = 4$), lo que introduce más incertidumbre de la deseable en una estimación que, idealmente, se ajustaría con más réplicas y más niveles de N sobre un clúster real. El ajuste de mínimos cuadrados sobre los tres puntos coincidió con el cálculo algebraico puntual dentro de 0.2 %, lo que da cierta confianza en la estimación, pero no elimina esta limitación de fondo.

Un tercer punto, ya anticipado en el fundamento teórico, es la distinción entre la fracción serial *algorítmica* de las cinco transformaciones (que, examinado el código, es casi nula: ninguna transformación tiene una sección explícitamente marcada como no paralelizable) y la fracción no escalable *observada* de 46 %, que es enteramente overhead de coordinación y sobrecarga del motor. Esto significa que la Ley de Amdahl, aplicada de forma ingenua a la fracción de código no paralelizable, predeciría un *speedup* mucho más alto del observado; el modelo sigue siendo válido matemáticamente, pero la variable de entrada (p) que se le alimenta en la práctica captura mucho más que el algoritmo en sí. Esta es, en opinión del equipo, la lección conceptual más importante de esta práctica: la Ley de Amdahl no falla al predecir un *speedup* bajo, sino que revela –correctamente– que la mayor parte del «trabajo» de este *pipeline* a esta escala es coordinación distribuida, no cómputo paralelizable.

Finalmente, desde la perspectiva del dominio de negocio (Soporte Técnico ISP), estos resultados tienen una implicación directa y quizás contraintuitiva: si el volumen real de tickets de un ISP pequeño o mediano se mantuviera en el orden de cientos de miles de registros por lote de análisis, la recomendación técnica honesta –respaldada por la medición, no por la expectativa– sería *no* migrar a Spark, y mantener el análisis en pandas hasta que el volumen justifique el costo de coordinación. La migración a un motor distribuido solo se justifica, para este dominio, a partir del umbral de volumen que se cuantifica en la Pregunta 2 del Anexo A, o cuando el patrón de

acceso deje de ser por lotes y pase a streaming continuo, escenario abordado en la Pregunta 3.

6. Anexo A — Preguntas de análisis

Pregunta 1. Speedup teórico máximo de T3 y N para el 90 % de $S_{\text{máx}}$

Con la fracción no escalable observada para T3 (*join*), $p = 0,4607$, obtenida al sustituir $N = 4$ y el *speedup* medido $S(4) = 1,6792$ en la forma inversa de Amdahl (Equation (5)): $p = \left(\frac{1}{1,6792} - \frac{1}{4} \right) / \left(1 - \frac{1}{4} \right) = \frac{0,5956-0,25}{0,75} = 0,4607$ (desarrollo completo en Equation (6)), el *speedup* teórico máximo cuando $N \rightarrow \infty$ (Equation (2)) es $S_{\text{máx}} = 1/(1-p) = 1/(1-0,4607) = 1/0,5393 = 1,8543$ (Equation (8)). Este valor establece un techo absoluto: sin importar cuántos *executors* adicionales se agreguen al clúster, la transformación T3 –tal como está implementada y con la sobrecarga de coordinación medida en este entorno– nunca podrá ejecutarse más de 1,85 veces más rápido que en un único hilo.

Para determinar con cuántos procesadores se alcanza el 90 % de ese máximo ($0,9 \times 1,8543 = 1,6689$), se despeja N de $S(N) = 0,9 S_{\text{máx}}$ en la Ecuación (1), obteniendo la forma general $N = p / \left(\frac{1}{0,9 S_{\text{máx}}} - (1-p) \right)$ (Equation (9)). Sustituyendo: $N = 0,4607 / \left(\frac{1}{0,9 \times 1,8543} - 0,5393 \right) = 0,4607 / (0,5990 - 0,5393) = 0,4607 / 0,0599 \approx 7,69$ (Equation (10)). Es decir, se necesitarían **aproximadamente 8 *executors*** para que T3 alcance el 90 % de su *speedup* máximo posible. Dado que la fracción serial observada es relativamente alta (46 %), este número es comparativamente bajo –con una fracción serial menor, digamos $p = 0,9$, haría falta un N mucho mayor para alcanzar el mismo 90 % del máximo, porque el máximo mismo sería mucho más alto ($S_{\text{máx}} = 10$) y por tanto más difícil de aproximar–. Esta relación inversa entre qué tan alto es el techo y qué tan fácil es acercarse a él es una consecuencia directa de la forma hiperbólica de la Ecuación (1) [4].

Pregunta 2. Speedup menor que 1: overhead de Spark y tamaño mínimo de dataset rentable

Las cinco transformaciones de este trabajo mostraron *speedup* menor que 1 –PySpark más lento que pandas– porque a 600,000 filas el costo fijo de coordinar un motor distribuido supera el tiempo que tomaría simplemente calcular el resultado en un único proceso. Ese costo fijo (*overhead*) tiene varios componentes medibles indirectamente en este trabajo: el arranque de la JVM y de la sesión de Spark (varios segundos, fuera del cronómetro de cada transformación individual pero presente en el tiempo total de ejecución del *pipeline*); la planificación de *tasks* por el driver y su distribución a los *executors*; la serialización de datos entre el proceso Python y la JVM a través del puente Py4J, necesaria en cada llamada a una función de PySpark; y, para operaciones con *shuffle* como T2 y T3, la escritura y lectura de datos intermedios entre particiones, visible como nodos **Exchange** en el plan físico capturado en la Spark UI (Figure 5). Comparando directamente overhead absoluto por transformación (columna $T_{\text{PySpark}} - T_{\text{pandas}}$ de la Table 1), el rango va de 0,49s (T4) a 1,73s (T2), consistente con que T2 y T3 –las que requieren *shuffle*– son las más penalizadas proporcionalmente.

Para estimar el tamaño mínimo de dataset que justificaría usar Spark en este dominio, se extrapola linealmente el tiempo de pandas (que crece aproximadamente en proporción al número de filas, dado que T1 es un filtro de complejidad $O(n)$) y se asume que el overhead fijo de Spark permanece aproximadamente constante mientras su costo marginal por fila es menor que el de pandas (dado que Spark vectoriza y paraleliza el cómputo real, aunque no su coordinación). Con el costo por fila de pandas en T1, $0,1846\text{s}/600,000 \approx 3,08 \times 10^{-7}$ s/fila, y asumiendo –conservadoramente– que el costo marginal de Spark es aproximadamente la mitad ($1,5 \times 10^{-7}$ s/fila) sobre un overhead fijo estimado de 0,9s, el punto de cruce $T_{\text{pandas}}(n) = T_{\text{Spark}}(n)$ se estima en $n \approx 0,9 / (3,08 \times 10^{-7} - 1,5 \times 10^{-7}) \approx 5,7 \times 10^6$ filas –un orden de magnitud de varios millones de registros–. Esta cifra es una extrapolación, no una medición directa (este trabajo solo midió

un punto de volumen), pero es consistente con el orden de magnitud reportado empíricamente por Ahmed et al. para cargas de una sola pasada, en las que la ventaja de Spark frente a un motor de un solo nodo aparece recién en volúmenes grandes, y se vuelve mucho más marcada solo en cargas iterativas o de *machine learning* [10]. Para el dominio ACC, esto significa que un ISP con un volumen de tickets mensual por debajo de ese umbral debería mantener su análisis en pandas; la migración a Spark se justificaría al acumular varios años de histórico para análisis retrospectivos, o al pasar a un régimen de streaming continuo (Pregunta 3).

Pregunta 3. Rediseño con Spark Structured Streaming para datos en tiempo real

Si los tickets del ISP llegaran en tiempo real en vez de procesarse por lotes, el *pipeline* se rediseñaría sobre Spark Structured Streaming, que trata un flujo continuo como una tabla que crece indefinidamente y reutiliza la misma API de DataFrames que este trabajo ya usa en modo *batch* [2]. La **fente** (*source*) natural para eventos de creación de tickets sería un tópico de Apache Kafka (`readStream.format("kafka")`), donde cada evento de «ticket creado» o «ticket actualizado» se publica en el momento en que ocurre; alternatively, para una migración incremental más simple, podría usarse un directorio monitoreado de archivos (*file source*) si el sistema de origen los deposita periódicamente. Sobre ese flujo se aplicarían transformaciones equivalentes a T1 y T4 –filtrado y clasificación de prioridad– de forma *stateless*, evento por evento, reutilizando literalmente el mismo código de `t1_filtrado` y la UDF de `t4_columna_derivada` de este trabajo, ya que Structured Streaming ejecuta las mismas transformaciones de DataFrame sin cambios. La transformación T2 (agrupación), en cambio, requeriría una *watermark* explícita –por ejemplo, `withWatermark("ticket_created", "10 minutes")`– para acotar cuánto tiempo se espera a eventos tardíos antes de cerrar una ventana de agregación por estado, evitando que el estado acumulado crezca indefinidamente.

El **sink** de salida dependería del consumidor: para alimentar un panel de priorización de la cola de soporte en tiempo casi real, un *sink* a una base de datos operacional (por ejemplo, con el conector JDBC de Spark hacia una base distribuida como CockroachDB) permitiría que el sistema de atención al cliente consulte la prioridad actualizada; para análisis histórico acumulativo, un *sink* a almacenamiento en formato Parquet particionado por fecha preservaría los datos crudos para reprocesar. La **política de triggers** –con qué frecuencia se materializa un nuevo micro-lote– se configuraría con `trigger(processingTime="1 minute")` para el caso de uso de priorización de soporte, donde una latencia de un minuto es aceptable y preferible a la sobrecarga de un modo *continuous* de latencia sub-segundo, reservado para casos donde cada milisegundo importa. En cuanto a **garantías de entrega**, Structured Streaming ofrece semántica *exactly-once* extremo a extremo combinando *checkpointing* del progreso de lectura (offsets de Kafka) con *sinks* idempotentes; para el *sink* JDBC esto exige diseñar la escritura como un *upsert* por `ticket_id` en vez de un *insert* simple, de modo que un reprocesamiento tras una falla –recuperado gracias al mismo *lineage* reproducible que sustenta la tolerancia a fallos de los RDD subyacentes– no duplique registros [1].

Pregunta 4. Por qué Spark es más eficiente que MapReduce para procesamiento iterativo

La ventaja de Spark sobre MapReduce en cargas iterativas –entrenamiento de modelos de aprendizaje automático, algoritmos de grafos, o cualquier flujo que recorre el mismo conjunto de datos varias veces– se explica directamente por dos decisiones de diseño complementarias: el *lineage* de los RDD y la evaluación perezosa. En el modelo MapReduce clásico, cada paso *map/reduce* lee su entrada desde HDFS y escribe su salida de vuelta a HDFS, es decir, a disco [8], [20]. Un algoritmo iterativo de k pasos sobre el mismo conjunto de datos –por ejemplo, k iteraciones de descenso de gradiente sobre el mismo conjunto de entrenamiento– implica, en

MapReduce puro, k ciclos completos de lectura y escritura a disco del conjunto completo, incluso si ese conjunto cabe íntegramente en la memoria RAM del clúster. El costo de I/O a disco, uno o dos órdenes de magnitud más lento que el acceso a memoria, domina el tiempo total y crece linealmente con el número de iteraciones.

Spark evita esto manteniendo los datos como RDD –o DataFrames– **en memoria** entre transformaciones sucesivas, cacheando explícitamente (`.cache()`/`.persist()`) los conjuntos que se reutilizan en múltiples iteraciones. La pregunta natural es cómo se logra tolerancia a fallos sin la replicación física a disco que ofrece HDFS: la respuesta es el *lineage*, el grafo de transformaciones que produjo cada RDD a partir de los datos de origen [1]. Si una partición se pierde –por ejemplo, por la caída de un *executor*– Spark no necesita leer una copia de respaldo desde disco, como haría HDFS; simplemente recomputa esa partición reaplicando su *lineage* sobre los datos de origen (que sí están replicados de forma duradera). Esto convierte la tolerancia a fallos en un costo que solo se paga cuando efectivamente ocurre una falla, en vez de un costo fijo de escritura replicada en cada paso, como en MapReduce.

La **evaluación perezosa** complementa esta ventaja de un modo menos obvio pero igual de importante: como las transformaciones no se ejecutan hasta que una acción las requiere, Spark puede observar la cadena completa de transformaciones antes de ejecutar nada y optimizarla como un todo –por ejemplo, fusionando varias transformaciones `map` consecutivas en un único paso de `WholeStageCodegen`, como se observó directamente en el plan físico de T3 de este trabajo (Figure 5)–, en vez de materializar un resultado intermedio en disco después de cada paso individual, como fuerza la arquitectura de MapReduce. Zaharia et al. reportan mejoras de uno a dos órdenes de magnitud de Spark sobre Hadoop MapReduce específicamente en cargas iterativas de aprendizaje automático, mientras que la ventaja es mucho más modesta –o, como este mismo trabajo demuestra, inexistente– en cargas de una sola pasada sobre datasets de tamaño moderado, exactamente la asimetría que predice este razonamiento [1].

7. Conclusiones individuales

Jeremy Alexis Alvarez Parraga

El desarrollo de esta práctica me permitió comprender que utilizar una tecnología distribuida no significa automáticamente obtener un mejor rendimiento. Antes de realizar las pruebas, podía parecer lógico pensar que PySpark sería más rápido que pandas por estar diseñado para trabajar de forma paralela; sin embargo, los resultados mostraron una realidad diferente para el volumen de datos utilizado. Esto me ayudó a entender la importancia de medir antes de tomar una decisión tecnológica y no basarse únicamente en las características que una herramienta promete ofrecer. También considero importante haber trabajado con las mismas transformaciones en pandas y PySpark, porque esto permitió comparar ambos enfoques bajo condiciones similares y comprobar que los resultados obtenidos fueran equivalentes. Desde el punto de vista del sistema de Soporte Técnico ISP, este aprendizaje resulta útil porque una empresa no debería incorporar una solución más compleja si el volumen actual de información todavía puede procesarse de manera adecuada con herramientas más sencillas. En mi opinión, la principal enseñanza de la práctica fue aprender que una buena decisión en ingeniería no consiste en utilizar siempre la tecnología más potente, sino en seleccionar la que realmente se adapte a las necesidades y condiciones del problema.

Jhinson Stalyn Aucatoma Celorio

Esta práctica me ayudó a comprender de mejor forma la relación entre lo que se estudia de forma teórica sobre paralelismo y lo que realmente puede observarse durante una ejecución. La Ley de Amdahl no solo es una fórmula para calcular *speedup*, porque al comparar los tiempos de pandas y PySpark fue posible observar que existen partes del procesamiento que no mejoran simplemente por utilizar más recursos. También me pareció importante comprobar que el costo

de preparar, coordinar y ejecutar el trabajo puede llegar a tener una influencia considerable en el resultado final. Uno de los aspectos que más destaco de esta práctica es que los resultados no fueron los que inicialmente se podrían esperar de Spark, pero precisamente eso permitió analizar con mayor profundidad lo ocurrido en lugar de limitar la práctica a demostrar que una tecnología era más rápida que otra. Relacionándolo con el PFC de Soporte Técnico ISP, considero que este tipo de evaluación serviría para decidir en qué momento el crecimiento de los registros de incidencias realmente justificaría cambiar la forma en que se procesan los datos

Carlos Jose Carpio Mendoza

A partir de esta práctica comprendí mejor la importancia de realizar experimentos que puedan repetirse y compararse de manera justa. No fue suficiente con ejecutar una operación una sola vez y observar cuánto demoraba, sino que fue necesario realizar varias mediciones, descartar una ejecución inicial y utilizar la mediana para representar mejor el comportamiento de cada transformación. Esto me permitió notar que, cuando se analiza rendimiento, pequeñas diferencias en la forma de medir pueden llevar a conclusiones equivocadas. Otro aspecto que considero valioso fue comprobar que pandas y PySpark entregaran resultados equivalentes, porque una implementación más rápida o más lenta pierde sentido si no produce la misma información. En el contexto de Soporte Técnico ISP, esto se relaciona directamente con la confiabilidad de los análisis que podrían realizarse sobre tickets, tipos de incidencias o prioridades de atención, ya que cualquier decisión basada en esos datos necesita partir de resultados correctos. Finalmente, considero que la práctica permitió combinar programación, análisis de datos y evaluación de rendimiento de una manera más cercana a un problema real. Mi principal aprendizaje fue que antes de recomendar una solución tecnológica se debe comprobar tanto que funcione correctamente como que su uso aporte una ventaja que realmente justifique su implementación.

Bibliografía

- [1] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauly, M. J. Franklin, S. Shenker e I. Stoica, «Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing,» en *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI '12)*, San Jose, CA, USA: USENIX Association, 2012, págs. 15-28.
- [2] M. Armbrust, R. S. Xin, C. Lian, Y. Huai, D. Liu, J. K. Bradley, X. Meng, T. Kaftan, M. J. Franklin, A. Ghodsi y M. Zaharia, «Spark SQL: Relational Data Processing in Spark,» en *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data (SIGMOD '15)*, Melbourne, VIC, Australia: ACM, 2015, págs. 1383-1394. DOI: [10.1145/2723372.2742797](https://doi.org/10.1145/2723372.2742797).
- [3] S. Tang, B. He, C. Yu, Y. Li y K. Li, «A Survey on Spark Ecosystem: Big Data Processing Infrastructure, Machine Learning, and Applications,» *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, n.º 1, págs. 71-91, 2022. DOI: [10.1109/TKDE.2020.2975652](https://doi.org/10.1109/TKDE.2020.2975652).
- [4] G. M. Amdahl, «Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities,» en *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference (AFIPS '67 (Spring))*, Atlantic City, NJ, USA: ACM, 1967, págs. 483-485. DOI: [10.1145/1465482.1465560](https://doi.org/10.1145/1465482.1465560).
- [5] J. L. Gustafson, «Reevaluating Amdahl's Law,» *Communications of the ACM*, vol. 31, n.º 5, págs. 532-533, 1988. DOI: [10.1145/42411.42415](https://doi.org/10.1145/42411.42415).
- [6] A. Grama, A. Gupta, G. Karypis y V. Kumar, *Introduction to Parallel Computing*, 2.ª ed. Addison-Wesley, 2003, ISBN: 978-0-201-64865-2.
- [7] A. H. Karp y H. P. Flatt, «Measuring Parallel Processor Performance,» *Communications of the ACM*, vol. 33, n.º 5, págs. 539-543, 1990. DOI: [10.1145/78607.78614](https://doi.org/10.1145/78607.78614).
- [8] T. White, *Hadoop: The Definitive Guide*, 4.ª ed. Sebastopol, CA, USA: O'Reilly Media, 2015, ISBN: 978-1-4919-0163-2.
- [9] V. K. Vavilapalli, A. C. Murthy, C. Douglas, S. Agarwal, M. Konar, R. Evans, T. Graves, J. Lowe, H. Shah, S. Seth, B. Saha, C. Curino, O. O'Malley, S. Radia, B. Reed y E. Baldeschwieler, «Apache Hadoop YARN: Yet Another Resource Negotiator,» en *Proceedings of the 4th Annual Symposium on Cloud Computing (SoCC '13)*, Santa Clara, CA, USA: ACM, 2013, 5. DOI: [10.1145/2523616.2523633](https://doi.org/10.1145/2523616.2523633).
- [10] N. Ahmed, A. L. C. Barczak, T. Susnjak y M. A. Rashid, «A Comprehensive Performance Analysis of Apache Hadoop and Apache Spark for Large Scale Data Sets Using HiBench,» *Journal of Big Data*, vol. 7, n.º 1, 110, 2020. DOI: [10.1186/s40537-020-00388-5](https://doi.org/10.1186/s40537-020-00388-5).
- [11] Q. H. Khuat, «Mastering Apache Spark Architecture: A Guide to Optimizing Data Processing Workflows,» *World Journal of Advanced Engineering Technology and Sciences*, vol. 15, n.º 1, págs. 910-923, 2025. DOI: [10.30574/wjaets.2025.15.1.0294](https://doi.org/10.30574/wjaets.2025.15.1.0294).
- [12] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker e I. Stoica, «Spark: Cluster Computing with Working Sets,» en *Proceedings of the 2nd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud '10)*, USENIX Association, 2010.
- [13] T. Tekdogan y A. Cakmak, «Benchmarking Apache Spark and Hadoop MapReduce on Big Data Classification,» en *Proceedings of the 2021 5th International Conference on Cloud and Big Data Computing (ICCBDC '21)*, Liverpool, United Kingdom: ACM, 2021, págs. 15-20. DOI: [10.1145/3481646.3481649](https://doi.org/10.1145/3481646.3481649).
- [14] F. D. Utami y F. D. Astuti, «Comparison of Hadoop MapReduce and Apache Spark in Data Processing with HGrid247-DE,» *Journal of Applied Informatics and Computing (JAIC)*, vol. 8, n.º 2, págs. 390-399, 2024. DOI: [10.30871/jaic.v8i2.8557](https://doi.org/10.30871/jaic.v8i2.8557).

- [15] P. Mell y T. Grance, «The NIST Definition of Cloud Computing,» National Institute of Standards y Technology, Gaithersburg, MD, USA, Special Publication 800-145, 2011. DOI: [10.6028/NIST.SP.800-145](https://doi.org/10.6028/NIST.SP.800-145).
- [16] N. Poggi, A. Montero y D. Carrera, «Characterizing BigBench Queries, Hive, and Spark in Multi-cloud Environments,» en *Technology Conference on Performance Evaluation and Benchmarking (TPCTC 2017)*, ép. Lecture Notes in Computer Science, vol. 10661, Springer, 2017. DOI: [10.1007/978-3-319-72401-0_5](https://doi.org/10.1007/978-3-319-72401-0_5).
- [17] H. B. Hassan, S. A. Barakat y Q. I. Sarhan, «Survey on Serverless Computing,» *Journal of Cloud Computing: Advances, Systems and Applications*, vol. 10, 39, 2021. DOI: [10.1186/s13677-021-00253-7](https://doi.org/10.1186/s13677-021-00253-7).
- [18] M. Abughazala, *Architecting Data-Intensive Applications: From Data Architecture Design to Its Quality Assurance*, 2024. DOI: [10.48550/arXiv.2401.12011](https://doi.org/10.48550/arXiv.2401.12011). arXiv: [2401.12011](https://arxiv.org/abs/2401.12011).
- [19] G. Ganachari, «Lambda and Kappa Architectures for Data Processing in Healthcare Analytics,» *International Journal of Science and Research (IJSR)*, 2020. DOI: [10.21275/SR24801090044](https://doi.org/10.21275/SR24801090044).
- [20] J. Dean y S. Ghemawat, «MapReduce: Simplified Data Processing on Large Clusters,» *Communications of the ACM*, vol. 51, n.º 1, págs. 107-113, 2008. DOI: [10.1145/1327452.1327492](https://doi.org/10.1145/1327452.1327492).

Declaración de uso de inteligencia artificial generativa

Se utilizó Claude (Anthropic) como herramienta de apoyo durante la elaboración de este informe, no como autor del contenido técnico. Su uso se concentró en:

- Verificación de las referencias bibliográficas citadas en el fundamento teórico, comprobando que los DOI e ISBN correspondieran a las fuentes citadas y descartando editoriales depredadoras o fuentes no arbitradas.
- Apoyo en la redacción de las secciones de fundamento teórico, análisis de resultados y conclusiones individuales, a partir del contenido previamente elaborado por los integrantes del equipo. La herramienta se empleó únicamente para mejorar la coherencia, fluidez y conexión entre oraciones, sin introducir afirmaciones, datos o interpretaciones nuevas.
- Revisión puntual de la consistencia gramatical general del documento.

Las ideas, los hallazgos, la interpretación de los resultados y las conclusiones fueron elaborados originalmente por los integrantes del equipo a partir de su experiencia durante el desarrollo de la práctica y de la evidencia experimental obtenida. En ningún momento la IA generó por sí sola resultados experimentales, código de implementación ni las cifras utilizadas en el análisis comparativo de rendimiento.